Behavioral Research Workflow Workbook

A hands-on companion to the workshop deck – setup, prompt templates, and troubleshooting

Stephen Shu

2026

Quick Start: the whole flow in one page

If you only read one page before opening a terminal, read this one. Everything here is expanded later.

1. **Set up once**: VS Code + Claude Code, Python venv, git/GitHub, pandoc (Section 1).
2. **Scaffold a project**: the folder layout in Section 2 – one `projects/<slug>/` per study.
3. **Stage 3 (design)**: give Claude your control stimulus, treatment stimulus, population description, and outcome variable. Get back a structured design package.
4. **Stage 4 (synthetic test)**: pick a total N and batch size (10 is a safe default), run it, get back `subjects_data.csv`.
5. **Stage 5 (analysis)**: balance check -> regression -> chart. Read the balance report before trusting the regression.
6. **Stage 7 (writeup)**: assemble Method/Balance/Results into a Word doc, optionally benchmarked against a published study.

Worked example at a glance (the workshop's own case study, so you know what a completed run looks like):

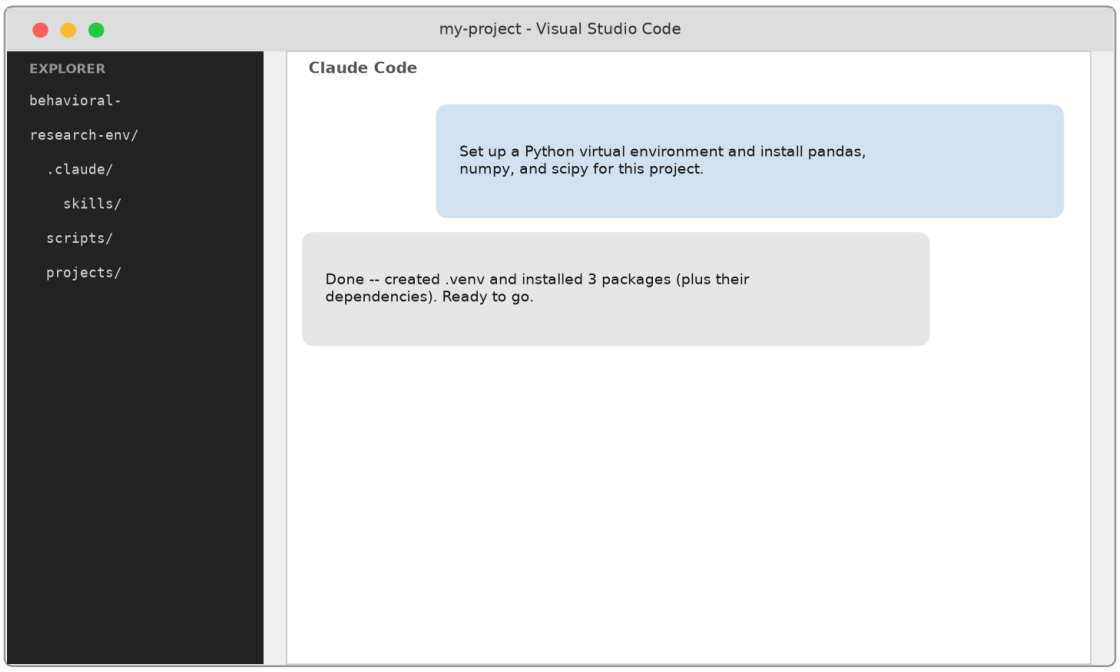
Step	What happened
Design	"$5 a day" (treatment) vs. "$150 a month" (control) savings framing; primary outcome = sign-up decision
Pilot	N=20, blinded; 80% vs. 40% sign-up, directionally large but only marginal (p~.08)
Scale-up	N=400 after unblinding; 64.5% vs. 45.5% sign-up, B=0.82, p<.001
Writeup	Compared against a real published study: savings-decision effect replicated, affordability effect replicated at about half the size, understandability effect did not replicate

That last row is the point worth remembering: a synthetic run can look convincing and still miss part of the real psychology. Treat it as a fast way to iterate on a design, not a replacement for real data.

1. Prerequisites & Setup

This section gets you from a bare machine to a working environment: VS Code, Claude Code, Python, and git/GitHub.

In plain English: install VS Code and the Claude Code extension, then tell Claude Code what you need – it runs the actual setup commands for you.



Illustrative mockup -- not an actual screenshot

Everything from here through the end of this section is the **"under the hood"** detail: the exact commands Claude Code runs on your behalf. You don't need to type any of it yourself – it's here so you understand what's happening, and so you have it if something needs troubleshooting.

1.1 Install VS Code and Claude Code *(under the hood)*

1. Install **VS Code** from its official source.
2. Install the **Claude Code** extension (or CLI) and sign in.
3. Open a folder in VS Code – this becomes your project's working directory.

1.2 Install Python *(under the hood)*

Check whether Python is already installed (same command on both platforms):

```
python --version
```

If it's missing, install it:

Windows (Windows sometimes shows a Microsoft Store stub instead of a real interpreter):

```
winget install Python.Python.3.12
```

Gotcha (Windows): after installing anything with winget, PATH changes do not appear in your *current* terminal session automatically. Refresh it manually, or open a new terminal window:

```
$machinePath = [System.Environment]::GetEnvironmentVariable("Path","Machine")
$userPath = [System.Environment]::GetEnvironmentVariable("Path","User")
$env:Path = $machinePath + ";" + $userPath
```

macOS (via Homebrew – see `brew.sh` if you don't have it yet):

```
brew install python@3.12
```

Gotcha (macOS): this gotcha is about Homebrew itself, not Python – if commands you just installed aren't found right after *installing Homebrew*, its own bin directory (`/opt/homebrew/bin` on Apple Silicon, `/usr/local/bin` on Intel) isn't on your `PATH` yet. The Homebrew installer prints an `eval "$(brew shellenv)"` line to add to `~/.zshrc`; add it, then open a new terminal or run `source ~/.zshrc`.

1.3 Create a project-local virtual environment *(under the hood)*

Windows (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r scripts/requirements.txt
```

macOS (Terminal):

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r scripts/requirements.txt
```

(`python3` because a bare `python` isn't guaranteed to exist on macOS. Alternatively, `source .venv/bin/activate` once, then just call `python`/`pip` directly for the rest of the session.)

A minimal `requirements.txt` for this kind of workflow:

```
pandas
numpy
scipy
statsmodels
matplotlib
openpyxl
python-dotenv
tabulate
```

(Add `anthropic` only if you plan to call the Claude API directly from a script, rather than using Claude Code subagents.)

1.4 Install git and set up GitHub *(under the hood)*

Windows (PowerShell):

```
winget install --id Git.Git
winget install --id GitHub.cli --source winget --scope user
```

Gotcha (Windows): some winget installers try to trigger an admin-elevation (UAC) prompt, which silently fails in a non-interactive terminal. Adding `--scope user` installs to your user profile instead and avoids the prompt entirely.

macOS (Terminal):

```
brew install git gh
```

Gotcha (macOS): macOS ships no `git` binary until either the Xcode Command Line Tools or Homebrew's own git are installed. If running `git` for the first time pops up a "Install Command Line Tools" prompt, accept it and wait for it to finish – or install git via Homebrew above to skip that prompt entirely.

Authenticate GitHub (this step is interactive – run it yourself in a real terminal, not through an automation tool):

```
gh auth login
```

Then create and push your repo:

```
git init
git add .
git commit -m "Initial commit"
gh repo create <repo-name> --private --source=. --remote=origin --push
```

1.5 Install pandoc (for generating docx/pptx/pdf output) *(under the hood)*

Windows (PowerShell):

```
winget install --id JohnMacFarlane.Pandoc --scope user
```

PDF conversion additionally needs a rendering engine. A lightweight option:

```
winget install --id Typst.Typst --scope user
```

macOS (Terminal):

```
brew install pandoc typst
```

Then convert with (same command on both platforms):

```
pandoc mydoc.md -o mydoc.pdf --pdf-engine=typst
```

Gotcha: if your markdown embeds images by relative path, pandoc resolves paths relative to your *current directory*, not the markdown file's location. Fix with `--resource-path`, or by running pandoc from the file's own directory. The flag's syntax differs by platform: Windows uses `;`-separated, backslash paths (`--resource-path=".;path\to\assets"`); macOS/Linux use `:`-separated, forward-slash paths (`--resource-path=".:path/to/assets"`).

---

2. Project Scaffolding

A recommended directory layout for this kind of workflow:

```
my-research-env/
  .claude/
    skills/
      behavioral-design/SKILL.md
      synthetic-ab-test/SKILL.md
      data-analysis/SKILL.md
      research-writeup/SKILL.md
  scripts/
    requirements.txt
    synthetic_sample/
      sample_population.py
      persona_prompt.py
      build_batches.py
      merge_responses.py
    analysis/
      balance_check.py
      regression.py
      charts.py
  projects/
    <study-slug>/
      03_design/
        research_design.md
        variables_spec.md
        population_spec.md
        stimuli/
          control.md
          treatment.md
      04_synthetic_test/
        population_config.json
        test_config.json
        subjects_frame.csv
        subjects_data.csv
      05_analysis/
        balance_report.md
        regression_results_*.md
        charts/*.png
      07_writeup/
        writeup.md
        writeup.docx
  .gitignore
```

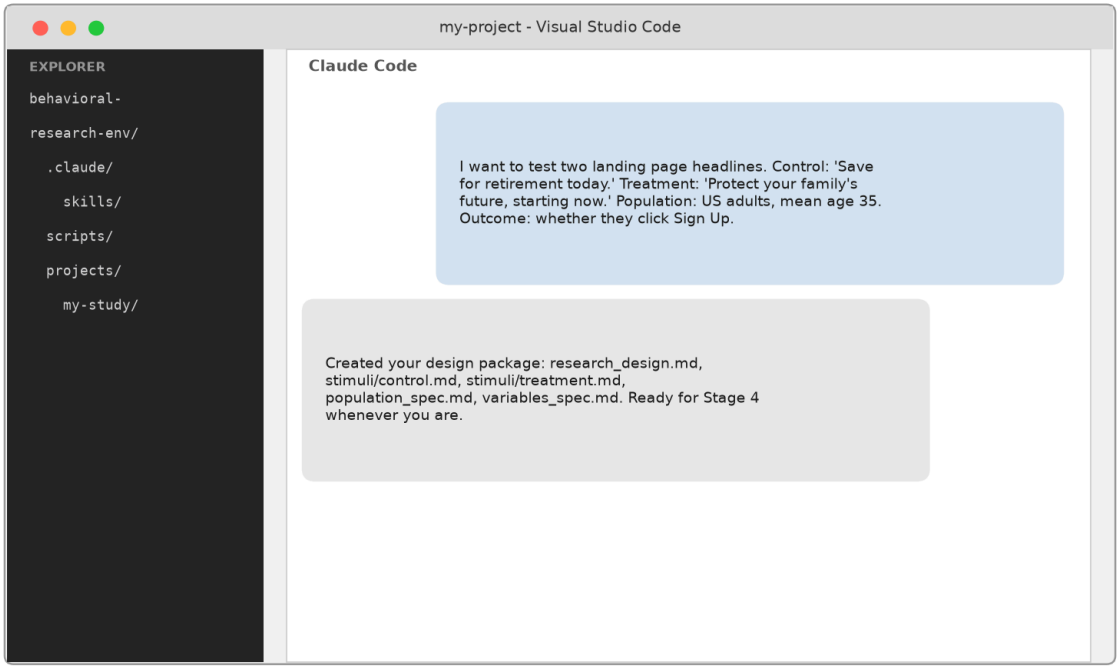
A `.gitignore` that keeps machine-specific and secret-bearing files out of version control:

```
.venv/
__pycache__/
*.pyc
.env
```

3. Stage-by-Stage Guide

Each stage below is a Claude Code **Skill** – a markdown instruction file the AI agent follows. You invoke a stage by describing what you want in plain language; Claude Code figures out which skill applies (or you can name it directly, e.g. "run behavioral-design").

3.1 Stage 3 – Behavioral Design (intake)



Illustrative mockup -- not an actual screenshot

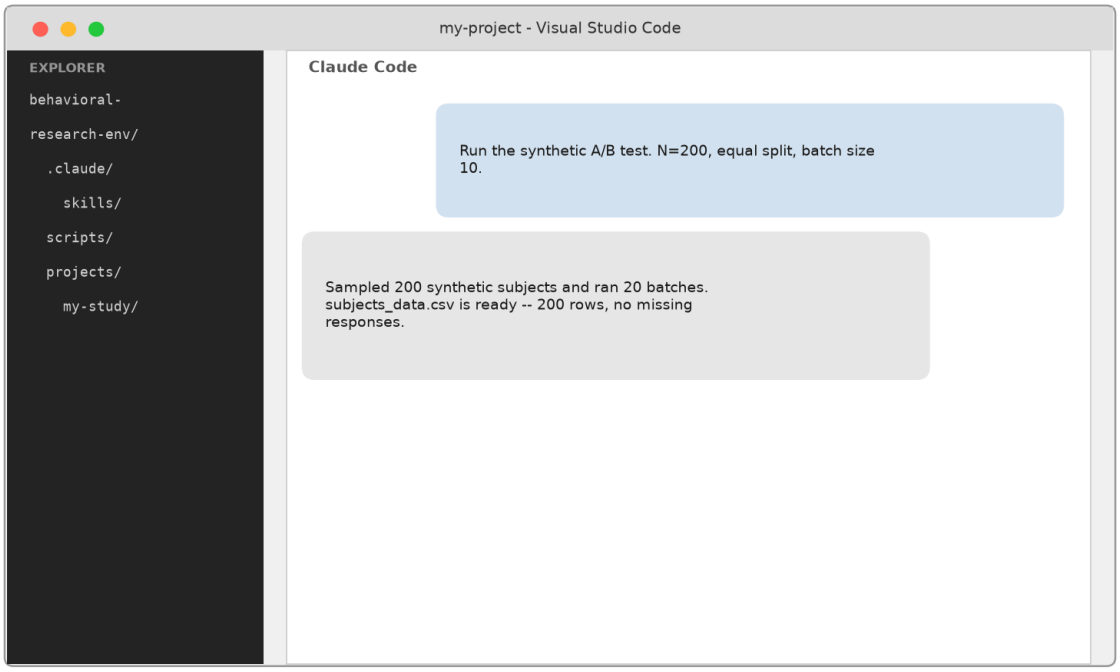
What it does: takes your control stimulus, treatment stimulus, population description, and outcome variable(s), and turns them into a structured design package Stage 4 can consume.

Copy-paste prompt template:

> I want to set up a behavioral test.
>
> Control condition stimulus: [PASTE CONTROL STIMULUS TEXT]
>
> Treatment condition stimulus: [PASTE TREATMENT STIMULUS TEXT]
>
> Population to sample: [DESCRIBE DEMOGRAPHICS AND INDIVIDUAL DIFFERENCES – e.g. age mean/SD, gender split, income, education, financial literacy, numeracy – with means/SDs if you have them, or a rough text description if not]
>
> Outcome variable(s): [NAME AND TYPE – e.g. "a binary choice between X and Y" or "a 1-7 Likert rating of Z"]
>
> Please set this up as a new project called [PROJECT-SLUG].

What you get back: `research_design.md`, `stimuli/control.md` and `stimuli/treatment.md`, `population_spec.md`, `variables_spec.md` – review these before moving on, especially `variables_spec.md`'s note on which category should be coded 1 for any binary outcome.

3.2 Stage 4 – Synthetic A/B Test



Illustrative mockup -- not an actual screenshot

What it does: samples a synthetic subject population, randomly assigns them to conditions, and collects each subject's response by simulating them with Claude (subagents, batched) or the direct API.

Copy-paste prompt template:

> Let's run the synthetic A/B test for [PROJECT-SLUG].
>
> Total N: [N]. Allocation: [equal / specify counts per condition].
>
> Use subagent mode with batch size [10 is a good default – smaller batches are more reliable per-subject, larger batches are more token-efficient].

What you get back: `subjects_frame.csv` (sampled profiles + condition), `subjects_data.csv` (profiles + condition + responses) – spot-check a few rows before moving to analysis.

Under the hood: config file reference and a worked example
Everything below is what Claude Code writes and reads behind the scenes – useful if you want to inspect or hand-edit a config, not required to run the stage.

`population_config.json` field reference:

Field	Meaning
`conditions`	list of condition names
`n_total`	total sample size
`allocation`	`"equal"` or a dict of exact per-condition counts
`variables`	list of `{name, kind: continuous|categorical, role, mean/sd/min/max/round}` or `{categories, probabilities}`
`correlations`	optional list of `{var1, var2, rho}` pairs

`test_config.json` field reference:

Field	Meaning
`mode`	`"subagent"` (default, no API key) or `"api"` (opt-in, needs `ANTHROPIC_API_KEY`)
`conditions`	dict of condition name -> stimulus text
`outcome_variable`	`{name, type, options/scale_min/scale_max, description, reasoning_prompt, secondary: [...]}`
`batch_size`	subjects simulated per subagent call (default 10)
`agent_type`	which subagent type to use (default `general-purpose`)

Concrete example: a filled-in `population_config.json`

This is an abbreviated version of the config used for the workshop's own temporal reframing case study (N reduced for space):

```
{
  "conditions": ["condition_1", "condition_2"],
  "n_total": 400,
  "allocation": "equal",
  "variables": [
    {"name": "age", "kind": "continuous", "role": "demographic",
     "mean": 34.29, "sd": 7.24, "min": 18, "max": 85},
    {"name": "gender", "kind": "categorical", "role": "demographic",
     "categories": ["female", "male"], "probabilities": [0.477, 0.523]},
    {"name": "income_bracket", "kind": "continuous", "role": "demographic",
     "mean": 5.29, "sd": 3.2, "min": 1, "max": 16, "round": true},
    {"name": "financial_literacy", "kind": "continuous", "role":
"individual_difference",
     "mean": 2.36, "sd": 0.91, "min": 0, "max": 3, "round": true}
```

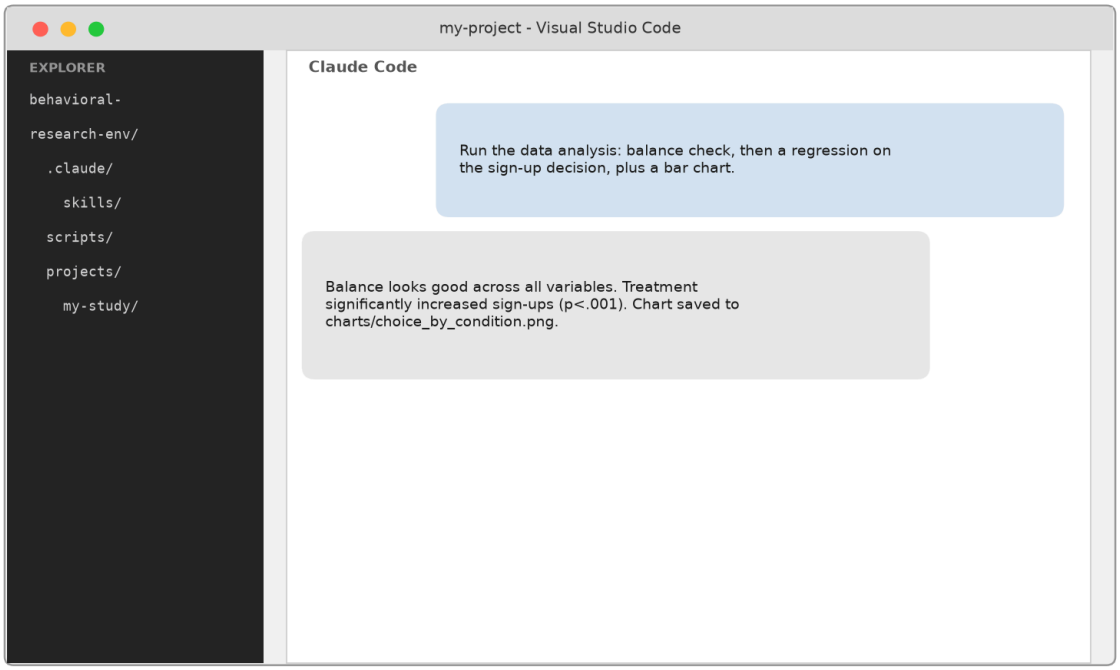
```
  ],
  "correlations": []
}
```

And the matching `test_config.json`:

```
{
  "mode": "subagent",
  "conditions": {
    "condition_1": "...get started with $5 a day today...",
    "condition_2": "...get started with $150 a month today..."
  },
  "outcome_variable": {
    "name": "choice",
    "type": "binary",
    "options": ["Not now", "Yes, start saving"],
    "description": "Whether the subject chooses to start saving.",
    "reasoning_prompt": "List three things you were thinking about, in your own
words."
  },
  "batch_size": 10,
  "agent_type": "general-purpose"
}
```

Notice `round: true` on `income_bracket` and `financial_literacy` – both are ordinal/count scales in the real instrument (a 16-point income bracket, a 0-3 correct-answer count), so fractional sampled values need rounding to stay meaningful.

3.3 Stage 5 – Data Analysis



Illustrative mockup -- not an actual screenshot

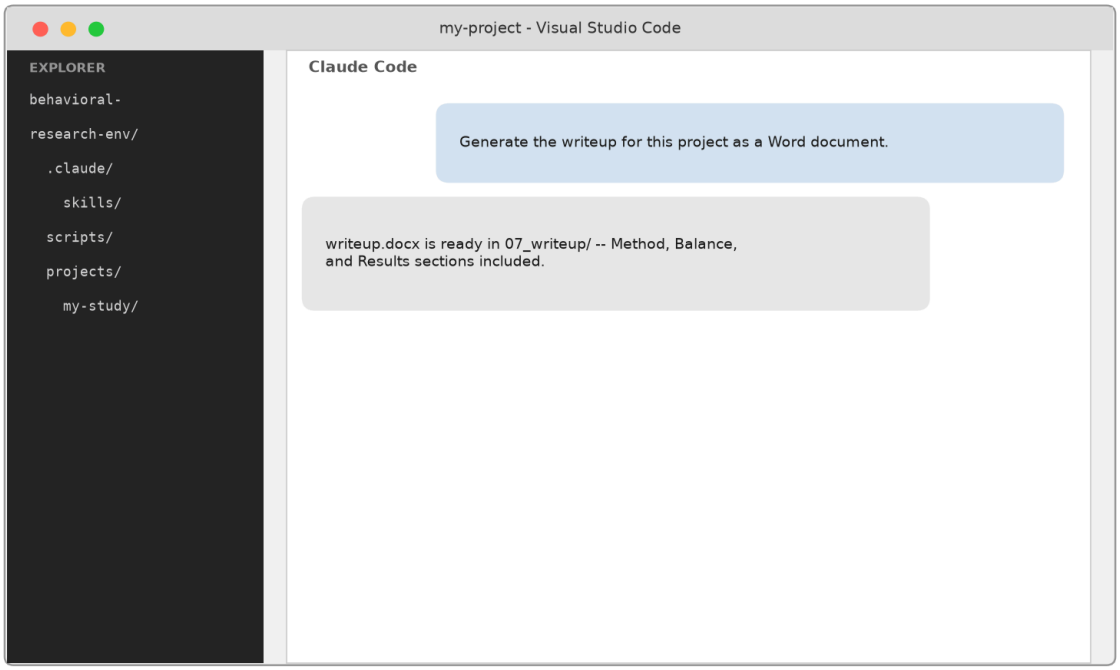
What it does: checks randomization balance, runs regressions on your outcome variable(s), and produces bar charts.

Copy-paste prompt template:

> Run the data analysis for [PROJECT-SLUG]:
>
> - Balance check across all sampled demographic/individual-difference variables
> - Regression on [OUTCOME VARIABLE NAME], type [binary/continuous], with controls: [none, or list variables]
> - If binary, code [WHICH CATEGORY] as 1
> - Bar chart of the outcome by condition

What you get back: `balance_report.md`, `regression_results_<outcome>.md`, `charts/<outcome>_by_condition.png` – read the balance report first; if anything is flagged, consider re-running the regression with that variable as a control.

3.4 Stage 7 – Research Writeup



Illustrative mockup -- not an actual screenshot

What it does: assembles Method, Balance, and Results sections into a Word-ready document, optionally compared against a reference study.

Copy-paste prompt template:

> Generate a Stage 7 writeup for [PROJECT-SLUG] (results-and-method scope, no mediation/moderation).
>
> [Optional] Compare against this reference study: [CITATION]. Reference statistics: [OUTCOME NAME: effect size, p-value, with/without controls – repeat per outcome you have a reference value for].

What you get back: `writeup.md` and `writeup.docx`. If you included a reference comparison, read the classification (replicates / partially replicates / does not replicate) critically – don't let a partial or non-replication get smoothed over in the prose.

---

4. Critical-Steps Checklist

- **Batch size is a real tradeoff.** Smaller batches (e.g. 2 subjects/call) cost roughly 4x more tokens per subject than larger batches (e.g. 10/call), because fixed prompt overhead (persona instructions, stimuli, schema) gets amortized across more subjects. But very large batches risk subjects' answers drifting toward each other. Start at 10 unless you have a reason not to.

- **Decide your blinding approach before Stage 4.** If you want to avoid analyst bias, ask for neutral condition labels (Condition A/B) and don't reveal which is treatment until after Stage 5 is complete.
- **When balance check flags an imbalance**, don't panic and don't ignore it either – re-run the regression with that variable as a control and confirm the main result holds. With many variables tested, some false positives are expected by chance (roughly a 1-in-4 chance of at least one $p<.05$ false positive across 6 independent tests at $\alpha=.05$).
- **Binary outcome coding is not automatic-safe.** Don't rely on an alphabetical default for which category is coded 1 – state it explicitly, since it determines the sign of every downstream effect.
- **Estimate token cost before scaling up.** At batch size 10, expect roughly 2,500-2,600 tokens per synthetic subject in subagent mode. For N subjects, that's very roughly `N x 2,600` tokens total, split across `N / batch_size` subagent calls.
- **A pilot run (small N) is worth doing before a full-scale run.** It catches configuration and prompt problems cheaply, and gives you a directional read even if it's not statistically definitive on its own.

4.1 Sizing a run: rough token/effort reference

These are rough, empirically-observed figures from subagent mode at batch size 10 – use them to sanity-check a run's cost/time before launching it, not as exact guarantees.

N (total subjects)	Batches (batch size 10)	Approx. tokens	Notes
20	2	~50,000	good for a quick pilot
100	10	~260,000	moderate live run, still manageable in one session
400	40	~1,040,000	a full-scale run; launch batches in parallel groups (e.g. 6-8 per message) to keep wall-clock time reasonable

For comparison, the same N in **API mode** (opt-in, direct Anthropic API calls, not batched) is roughly 600-800 tokens/subject – cheaper in raw tokens and billed separately in dollars rather than counting against your Claude Code usage, but requires its own API key and billing setup, and doesn't benefit from batching efficiency.

---

5. Troubleshooting Appendix

Symptom	Cause	Fix
`winget install` reports success then the tool still isn't found (Windows)	PATH updated on disk but not in your current shell process	Refresh `$env:Path` from Machine+User scope, or open a new terminal
`winget install` fails/cancels with an installer exit code around 1602 (Windows)	Installer tried to trigger a UAC elevation prompt that can't be answered non-interactively	Retry with `--scope user`
A `brew`-installed tool isn't found right after installing Homebrew itself (macOS)	Homebrew's own bin directory (`/opt/homebrew/bin` Apple Silicon, `/usr/local/bin` Intel) isn't on `PATH` yet	Add the `eval "$(brew shellenv)"` line Homebrew's installer prints to `~/.zshrc`, then open a new terminal or `source ~/.zshrc`
Running `git` for the first time pops up an "Install Command Line Tools" dialog (macOS)	macOS ships no `git` binary until Xcode Command Line Tools or Homebrew's git are installed	Accept the prompt and wait for it to finish, or `brew install git` to skip it
pandoc converts markdown but images are missing from the output	pandoc resolves relative image paths from the working directory, not the markdown file's folder	Add `--resource-path` (`;`-separated backslash paths on Windows, `:`-separated forward-slash paths on macOS/Linux), or `cd` into the markdown file's folder first
A regression script errors on a column that "should" be numeric	Newer pandas versions may store text as a `str` dtype instead of the classic `object` dtype, breaking `dtype == object` checks	Use `pandas.api.types.is_numeric_dtype(...)` instead of comparing dtype directly
A statistical test (e.g. Bartlett's) throws an obscure internal array error	Some scipy versions mishandle integer-dtype input in certain test implementations	Cast the column to float before passing it to the test
A subagent's JSON response has extra text before/after the array	Subagents don't have a forced-output guarantee the way a direct API tool-call does	Instruct explicitly to return *only* the JSON array with no surrounding text; strip/retry if it still wraps the output in a code fence
`git commit` succeeds but shows a warning about name/email being auto-detected	git had no configured identity and guessed one from your username/hostname	Set it explicitly: `git config --global user.name "..."` and `git config --global user.email "..."`; amend the commit with `--reset-author` if needed
A binary outcome's regression coefficient has the "wrong" sign	The default coding falls back to alphabetical order when no positive category is specified	Always pass the category that should count as 1 explicitly rather than relying on a default
`gh auth login` can't complete in a non-interactive session	The device-code/browser auth flow needs a human [illegible]	Run it yourself in a real terminal window, [illegible] cache/push [illegible] afterward using the resulting stored credentials

6. Glossary & Command Cheat Sheet

Synthetic subject – an LLM-simulated research participant, given a sampled demographic/individual-difference profile and asked to respond in character.

Batch – a group of synthetic subjects simulated within a single subagent call, to amortize fixed prompt overhead.

Condition – one arm of a between-subject experiment (e.g. control vs. treatment).

Subagent – an independent Claude Code conversation spawned to do a self-contained piece of work (here: simulate one batch of subjects) and return a result.

Blinding – withholding which condition is "treatment" vs. "control" from the analyst until after data collection, to prevent bias in analysis or interpretation.

Balance check – a statistical test confirming that random assignment produced similar-looking groups across conditions (age, gender, etc.), which is a precondition for trusting a simple between-condition comparison.

Skill – a markdown file (`SKILL.md`) that packages instructions for a recurring task, which Claude Code follows when the task matches.

Allocation – how a study's total N is split across conditions (e.g. `"equal"` for an even split, or exact per-condition counts).

Positive-label coding – the explicit choice of which category of a binary outcome is coded 1 (vs. 0), made explicit rather than left to an alphabetical default so the sign of every effect is interpretable.

Balance imbalance (false positive) – a statistically significant difference between conditions on a variable that should, in truth, be balanced by randomization; expected to occur by chance some fraction of the time across many tests, so it should prompt a robustness check rather than an assumption of a broken sampler.

Effect size – the magnitude of a treatment's impact (e.g. a regression coefficient), as distinct from its statistical significance (p-value); a result can be significant with a small effect size, or large but not significant at a small N.

Mediator / moderator (out of scope here) – a mediator is a variable that explains *why* an effect occurs (e.g. affordability perception explaining a framing effect); a moderator is a variable that changes *how strong* an effect is for different subgroups (e.g. financial literacy). Both require dedicated analyses (Query Theory / GSEM mediation, floodlight moderation) not included in this workflow's v1 scope.

Replication assessment – classifying a result against a reference study as *replicates* (same sign, both significant), *partially replicates* (same sign, only one significant), or *does not replicate* (opposite sign, or a near-zero estimate where the reference was significant).

Command cheat sheet

Windows (PowerShell):

```
# Refresh PATH in a new PowerShell session
$machinePath = [System.Environment]::GetEnvironmentVariable("Path","Machine")
$userPath = [System.Environment]::GetEnvironmentVariable("Path","User")
$env:Path = $machinePath + ";" + $userPath

# Create and populate a venv
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r scripts\requirements.txt

# Git + GitHub
git init
git add .
git commit -m "message"
gh repo create <name> --private --source=. --remote=origin --push
git push

# pandoc conversions
pandoc doc.md -o doc.docx
pandoc doc.md -o doc.pptx
pandoc doc.md -o doc.pdf --pdf-engine=typst --resource-path=".;assets"
```

macOS (Terminal):

```
# Pick up Homebrew's PATH in a new shell (if freshly installed)
source ~/.zshrc

# Create and populate a venv
python3 -m venv .venv
.venv/bin/python -m pip install -r scripts/requirements.txt

# Git + GitHub
git init
git add .
git commit -m "message"
gh repo create <name> --private --source=. --remote=origin --push
git push

# pandoc conversions
pandoc doc.md -o doc.docx
pandoc doc.md -o doc.pptx
pandoc doc.md -o doc.pdf --pdf-engine=typst --resource-path=".:assets"
```